

Task 2 Report

Cross-Platform Utility Application with Local Storage

Internship: App Development Internship

Task: Task 2

Application Name: Task Manager

Developer: Nora Nagiy Sayed Hafez

Technology: Flutter

Storage: Hive

State Management: BLoC / Cubit

1. Objective

The objective of this task was to develop a functional cross-platform mobile application capable of performing CRUD operations using a local data storage solution.

The application was designed as a Task Manager that allows users to create, view, update, delete, search, filter, and manage their tasks.

2. Application Overview

Task Manager is a mobile application developed using Flutter.

The application provides users with a simple interface for managing their daily tasks. Each task can contain a title, description, creation date, due date, and completion status.

All task data is stored locally on the device using Hive, allowing the application to work without requiring an internet connection.

3. Technologies Used

Flutter

Flutter was used to develop the cross-platform mobile application and build the user interface.

Dart

Dart was used as the main programming language.

Hive

Hive was used as the local NoSQL database for storing task data on the device.

SharedPreferences

SharedPreferences was used to save the user's selected Light/Dark Mode preference.

BLoC / Cubit

Cubit was used for state management and handling task operations between the UI, repository, and local database.

Material 3

Material 3 components were used to create a modern and responsive user interface.

4. Main Features

The application includes the following features:

- Create new tasks
 - View saved tasks
 - Edit existing tasks
 - Delete tasks
 - Mark tasks as completed
 - Set task due dates
 - Search tasks
 - Filter tasks
 - Light Mode
 - Dark Mode
 - Persistent theme preference
 - Responsive user interface
 - Local data persistence
-

5. CRUD Implementation

Create

Users can create a task by entering:

- Task title
- Task description
- Due date

After pressing the Add Task button, the task is stored in the local Hive database.

Read

All saved tasks are retrieved from Hive and displayed on the Home Screen.

The application loads the stored tasks when it starts, allowing users to continue using their previously saved data.

Update

Users can update an existing task by selecting the Edit button.

They can modify:

- Task title
- Description
- Due date

Users can also change the completion status of a task.

Delete

Users can delete tasks using the Delete button.

A confirmation dialog is displayed before permanently removing the task.

6. Local Data Storage

Hive was selected as the local database because it is lightweight, fast, and suitable for Flutter applications.

The application stores task objects locally in a Hive box.

Each task contains:

ID
Title
Description
Created Date
Due Date

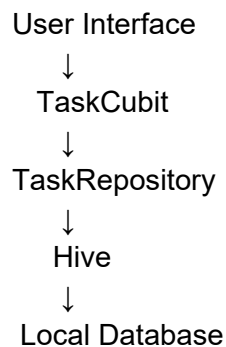
Completion Status

The stored data remains available after closing and reopening the application.

7. State Management

Cubit was used to manage the application state.

The general data flow is:



When a task is added, updated, or deleted, the Cubit updates the application state and the UI automatically reflects the changes.

8. Search and Filtering

The application provides a search feature that allows users to search for tasks using the task title or description.

The application also provides three filtering options:

- All
- Pending
- Completed

The number of tasks in each category is displayed to the user.

9. Theme Management

The application supports both Light Mode and Dark Mode.

Users can switch between the two modes using the button in the AppBar.

SharedPreferences is used to store the selected theme.

Therefore, when the application is closed and reopened, the previously selected theme is automatically restored.

10. Responsive UI

The application uses Flutter's responsive layout capabilities to provide a suitable interface for different screen sizes.

The layout adjusts its horizontal spacing and components depending on the available screen width.

This allows the application to work on mobile devices and larger screens.

11. User Interface

The application contains the following main screens and components:

Home Screen

The Home Screen displays:

- Total number of tasks
- Number of completed tasks
- Search bar
- Task filters
- Task list
- Add Task button
- Theme switch button

Add Task Screen

Users can enter task information and select a due date.

Edit Task

The same form is used to edit existing task information.

Delete Confirmation

A confirmation dialog is displayed before deleting a task.

12. Testing

The following operations were tested successfully:

Test	Result
Add Task	Passed
Display Tasks	Passed
Edit Task	Passed
Delete Task	Passed
Complete Task	Passed
Search Task	Passed
Filter Tasks	Passed
Set Due Date	Passed
Light Mode	Passed
Dark Mode	Passed
Save Theme Preference	Passed
Local Data Persistence	Passed
Responsive Layout	Passed

13. Challenges

During the development of the application, several challenges were addressed, including:

- Connecting the UI with local Hive storage.
- Managing task state using Cubit.
- Implementing CRUD operations.
- Updating the UI automatically after database changes.
- Maintaining task data after restarting the application.
- Implementing persistent Light/Dark Mode.
- Creating a responsive layout for different screen sizes.

These challenges helped improve my understanding of Flutter application architecture, local storage, and state management.

14. Conclusion

The Task Manager application successfully meets the main requirements of Task 2.

The application provides complete CRUD functionality, local data storage using Hive, state management using Cubit, responsive UI, search and filtering features, and persistent Light/Dark Mode.

This task provided practical experience in building a functional Flutter application and working with local databases and state management.